

# **Credit Card Approval Prediction**

Project Report

# 1. INTRODUCTION

## 1.1 Project Overview

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is time-consuming and prone to human error, making the process inefficient at scale. This project automates the credit card approval decision using machine learning, training and comparing four classification algorithms on historical applicant data, and deploying the best-performing model through a Flask web application for instant, real-time predictions.

## 1.2 Purpose

The purpose of this project is to replace slow, inconsistent manual review with a fast, data-driven system that predicts whether an applicant is likely to be a safe or risky borrower. This supports three groups: bank credit analysts screening new applications, compliance officers identifying high-risk applicants in bulk, and prospective customers checking their own eligibility before formally applying.

# 2. IDEATION PHASE

## 2.1 Problem Statement

| PS   | I am (Customer)       | I'm trying to                    | But                                  | Because                               | Which makes me feel    |
|------|-----------------------|----------------------------------|--------------------------------------|---------------------------------------|------------------------|
| PS-1 | A bank credit analyst | Decide approve or reject quickly | I get hundreds of applications daily | Manual review is slow and error-prone | Overwhelmed and behind |

## 2.2 Empathy Map Canvas

Persona: Bank Credit Analyst (reviews and decides on credit card applications)

### Says

- I have too many applications to review manually every day.
- I need more information before I can trust my decision.
- Some applicants look fine on paper but turn out risky later.

### Thinks

- There has to be a faster way to do this without missing risky cases.
- I wish I could just enter the details and get a reliable answer instantly.
- I'm worried about approving someone who ends up defaulting.

### Does

- Manually checks each applicant's income, employment, and credit history.
- Cross-references payment history records for red flags.
- Prioritizes which applications to review first based on gut feeling and available time.

### Feels

- Overwhelmed by the volume of applications.
- Anxious about making a wrong call that costs the bank money.
- Frustrated that manual review is slow and inconsistent between analysts.

## 2.3 Brainstorming

Step 1: Brainstorm and Idea Listing

| S.No | Idea / Suggestion                                                                                         | Category       | Group No. |
|------|-----------------------------------------------------------------------------------------------------------|----------------|-----------|
| 1    | Use ML to decide credit card approval instead of doing it by hand                                         | Core Idea      | 1         |
| 2    | Try 4 different models (Logistic Regression, Decision Tree, Random Forest, XGBoost) and pick the best one | Model Building | 2         |
| 3    | Change the payment status column into a simple yes/no risky label                                         | Data Cleaning  | 2         |

|   |                                                                        |             |   |
|---|------------------------------------------------------------------------|-------------|---|
| 4 | Make a website (Flask) where people can enter details and get a result | Application | 3 |
| 5 | Try hosting the model on IBM Watson so it can run on the cloud         | Deployment  | 3 |

Step 2: Idea Prioritization

| Group No. | Final Idea                                     | Feasibility | Importance | Priority | Selected |
|-----------|------------------------------------------------|-------------|------------|----------|----------|
| 1         | Use ML to automate the approval decision       | High        | High       | 1        | Yes      |
| 2         | Train 4 models and build the risky/safe label  | High        | High       | 1        | Yes      |
| 3         | Build the Flask app and try IBM Watson hosting | Medium      | High       | 2        | Yes      |

## 3. REQUIREMENT ANALYSIS

### 3.1 Customer Journey Map

Persona: Bank Credit Analyst reviewing a new credit card application.

| Stage                  | Actions                                              | Thoughts / Feelings                 | Opportunity                                 |
|------------------------|------------------------------------------------------|-------------------------------------|---------------------------------------------|
| 1. Receive application | Opens the new applicant's submitted form             | "Another one in the queue."         | Auto-collect data via web form              |
| 2. Gather information  | Manually checks income, employment, credit history   | "This is taking a while."           | Reduce manual data entry                    |
| 3. Assess risk         | Tries to judge if the applicant is likely to default | "I hope I'm not missing something." | Use ML model for consistent risk assessment |
| 4. Make decision       | Approves or rejects the application                  | "I hope this is the right call."    | Instant, data-driven prediction             |

### 3.2 Solution Requirement

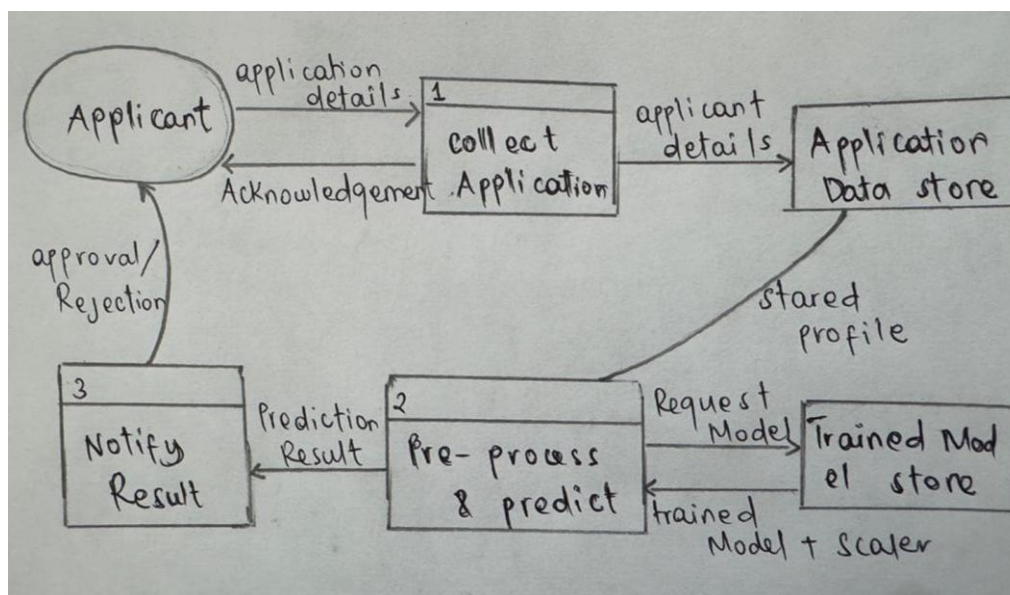
Functional Requirements (selected):

- Accept applicant details through a web form.
- Clean and preprocess applicant and credit history data.
- Train and compare at least four classification models.
- Select and save the best-performing model.
- Return an instant Approved/Rejected prediction with a risk category.

Non-Functional Requirements (selected):

- Prediction returned in under 2 seconds.
- Simple, easy-to-use interface for non-technical users.
- Model and preprocessing logic reusable without retraining per request.
- Extendable to cloud hosting (IBM Watson Machine Learning).

### 3.3 Data Flow Diagram



### 3.4 Technology Stack

| Component       | Technology          |
|-----------------|---------------------|
| Language        | Python 3.x          |
| Data Processing | Pandas, NumPy       |
| Visualization   | Matplotlib, Seaborn |

|                             |                             |
|-----------------------------|-----------------------------|
| Machine Learning            | Scikit-learn, XGBoost       |
| Web Framework               | Flask (HTML, CSS, Jinja2)   |
| Model Persistence           | Pickle                      |
| Cloud Deployment (optional) | IBM Watson Machine Learning |
| Version Control             | Git & GitHub                |

## 4. PROJECT DESIGN

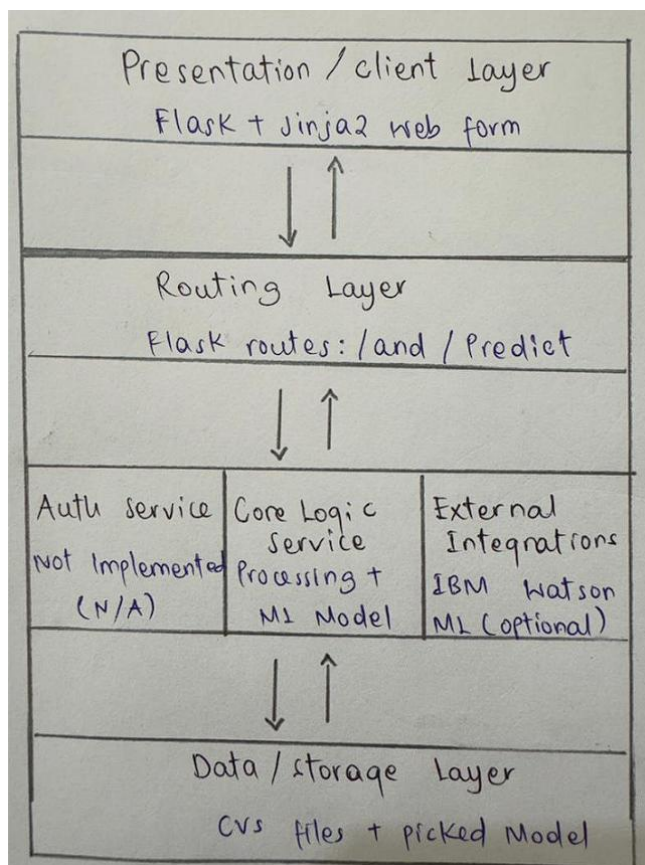
### 4.1 Problem Solution Fit

- Customer Segments: bank credit analysts, financial compliance officers, prospective customers.
- Problems/Pains: thousands of daily applications; slow, error-prone manual review.
- Root Cause: no fast, consistent, data-driven way to translate financial/credit history into a risk decision.
- Available Solutions: manual review (slow), rule-based scoring (too rigid), generic bureau scores (not tailored).
- Your Solution: an ML-powered web app that predicts approve/reject instantly from applicant data.

### 4.2 Proposed Solution

|              |                                                                                                                                     |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Objective    | Automate the credit card approval decision using machine learning, ensuring faster and more accurate assessments.                   |
| Scope        | Covers the full pipeline: data collection, preprocessing, model training/comparison, and deployment via a web application.          |
| Approach     | Employ machine learning to analyze applicant demographic, financial, and credit history data, predicting borrower risk.             |
| Key Features | ML-based credit risk model comparing 4 algorithms; near-instant decision-making via Flask; optional IBM Watson ML cloud deployment. |

### 4.3 Solution Architecture



## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

The project was broken into 5 epics, executed as 3 sprints between June 26 and July 4:

| Sprint   | Epic                    | Story Points | Start  | End    |
|----------|-------------------------|--------------|--------|--------|
| Sprint-1 | Data Collection         | 2            | 26-Jun | 28-Jun |
| Sprint-1 | Visualizing & Analysing | 6            | 26-Jun | 28-Jun |
| Sprint-2 | Data Pre-processing     | 9            | 29-Jun | 1-Jul  |
| Sprint-2 | Model Building          | 4            | 29-Jun | 1-Jul  |
| Sprint-3 | Application Building    | 6            | 2-Jul  | 4-Jul  |

## 6. FUNCTIONAL AND PERFORMANCE TESTING

### 6.1 Performance Testing

A lightweight load test was run against the live Flask application across four real scenarios:

| Scenario       | Virtual Users | Total Requests | Avg Response | Errors |
|----------------|---------------|----------------|--------------|--------|
| Light load     | 5             | 15             | 0.087 sec    | 0      |
| Moderate load  | 20            | 100            | 0.191 sec    | 0      |
| Heavy load     | 50            | 250            | 0.528 sec    | 0      |
| Home page load | 20            | 100            | 0.070 sec    | 0      |

Peak CPU utilization reached 40.3% and peak memory usage was approximately 161 MB during the heaviest test, both well within acceptable limits. All requests across all scenarios succeeded with a 0% error rate.

```
=== Load Test Results ===
Total requests sent: 100
Successful responses: 100
Errors: 0
Total wall time: 0.51 sec
Throughput: 195.77 req/sec

Avg response time: 0.255 sec
Max response time: 0.390 sec
Min response time: 0.073 sec
95th percentile: 0.384 sec

Error rate: 0.00%

Peak CPU utilization (Flask process): 40.3%
Peak memory usage (Flask process): 161.2 MB
```



## 7. RESULTS

### 7.1 Output Screenshots

Application form:

### Credit Card Approval Predictor

Fill in the applicant's details below to get an instant prediction.

**Gender**  
Male

**Age (years)**

**Owns a Car?**  
Yes

**Owns Realty/Property?**  
Yes

**Number of Children**  
0

**Number of Family Members**  
1

**Total Annual Income**

**Income Type**  
Working

**Education Type**  
Secondary / secondary special

**Family Status**  
Married

**Housing Type**  
House / apartment

**Occupation**  
Unknown / Not Specified

**Years Employed (0 if not currently employed)**  
0

Prediction result:

## Approved

Risk Category: **Low Risk**

[Check Another Applicant](#)

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Replaces slow manual review with an instant, consistent prediction.
- Compares four different algorithms rather than relying on a single model, ensuring the best available performer is used.
- Uses real historical data rather than arbitrary rule-based cutoffs, allowing the model to learn genuine risk patterns.

- Simple, accessible web interface usable by both technical and non-technical users.
- Verified end-to-end: tested with real safe and real risky applicant profiles, and load-tested with zero errors under simulated concurrent traffic.

### Disadvantages

- Severe class imbalance in the training data (only ~1.7% of applicants are labeled risky) limits precision - the model's risky-applicant predictions are correct only some of the time.
- No authentication or role-based access is implemented yet, so the app cannot currently distinguish between analyst, compliance officer, and customer users.
- Runs on Flask's development server locally; not yet deployed to IBM Watson Machine Learning or any production-grade hosting.
- No error handling for malformed or missing form input.
- The model predicts risk based on historical payment behavior as a proxy for approval, not a literal historical approve/reject label, since the dataset only contains applicants who were already issued a card.

## 9. CONCLUSION

This project successfully demonstrates an end-to-end machine learning pipeline for automating credit card approval decisions. Starting from raw applicant and credit history data, it cleans, merges, and engineers features from real-world data, trains and compares four classification algorithms, and deploys the best-performing model (Decision Tree) through an interactive Flask web application. The system was validated end-to-end, with the trained model correctly distinguishing between low-risk and high-risk applicant profiles based on real historical data, and the application verified to handle concurrent load without errors.

## 10. FUTURE SCOPE

- Improve handling of class imbalance using techniques like SMOTE or cost-sensitive ensemble learning.
- Incorporate more granular credit history features, such as average months overdue or trend over time.
- Complete IBM Watson Machine Learning cloud deployment for scalable hosting.
- Add user authentication and role-based access (Analyst / Compliance Officer / Customer).
- Add batch prediction support for CSV uploads, for bulk compliance screening.
- Add model explainability (e.g. SHAP) so analysts can see why an applicant was flagged as high-risk.

## 11. APPENDIX

| Item              | Details                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------|
| Source Code       | Included in the GitHub repository under "5. Project Development Phase".                            |
| Dataset Link      | Kaggle - Credit Card Approval Prediction (rikdifos): application_record.csv and credit_record.csv. |
| GitHub Repository | [Paste your GitHub repository link here]                                                           |
| Project Demo Link | [Paste your demo video link here, if recorded]                                                     |